

Project Explainer

Robust Solar Charging Hub — e-Micromobility Competition

What we built · Why every tool was chosen · What the results mean

20 kWp PV · 0 kWh · 8 chargers	£87,422/yr	96% guaranteed	16% (4.6 t/yr)
Best design	Cost saved (worst case)	Fleet service level	Carbon saved

1 — What Is This Project About?

Imagine a depot where dozens of electric scooters, e-bikes and cargo bikes need to be recharged every night. The depot has a solar panel on the roof and a limited electricity cable from the grid. The question is: **how big should the solar panel be, how large a battery do you need, and how many charge points should you install?**

The tricky part is that demand changes — a summer festival weekend is very different from a rainy Tuesday in January. If you build too small, vehicles can't charge and the fleet is stuck. If you build too large, you waste tens of thousands of pounds on equipment that sits idle. This project finds the **best design that stays good even in the worst plausible scenario** — what engineers call a *robust* solution.

The competition is run by the University of Warwick, IIT Kharagpur and IIT BHU Varanasi. It asks teams to design exactly this kind of solar charging hub using real data and engineering methods.

2 — The Real Data We Used

Three real-world datasets power every number in this project — nothing is made up:

- **DfT Newcastle e-scooter trial data** (Open Government Licence) — the UK Department for Transport's official monitoring data from the Newcastle trial: monthly trip counts, fleet size, deployment hours, trip distances. This is the demand foundation.
- **PVGIS solar data** — the European Commission's solar-irradiance API, queried for Newcastle (lat 54.978, lon -1.618). Gives hourly solar generation per kWp of panel for a full year — 8,760 hourly values.
- **National Grid ESO carbon-intensity API** — the real hour-by-hour carbon intensity (gCO₂/kWh) of the North East England electricity grid. Used to calculate actual carbon savings per hour, not just an annual average.

3 — The Problem (Why It Is Hard)

3.1 Uncertainty in demand

We do not know exactly how busy the fleet will be next year. We model three scenarios: **Low** (quiet), **Medium** (expected) and **High** (peak growth). To be realistic, each scenario is further split across three different year-on-year growth rates — giving **nine probability-weighted scenarios** in total.

3.2 The sizing dilemma

A design sized for Medium demand fails at High. A design sized for High demand wastes money at Low. Standard engineering picks the expected average — but that under-weights the correlated worst-case tail, which is exactly when all vehicles need charging at once.

3.3 The grid connection cap

The depot's grid connection is capped at **15 kW**. Exceeding it needs an expensive DNO upgrade. So the evening charging peak must come from on-site solar and battery, not just pulling more power from the street. This is a hard physical constraint the optimizer must respect.

4 — The Solution (Plain English)

We automated a computer to test **150 different station designs** — every combination of PV panel size (5 to 25 kWp), battery size (0 to 50 kWh) and number of charge bays (4 to 20). For each design it simulated an entire year hour by hour and calculated the cost and service level under all nine demand scenarios.

That gives a **150 × 9 cost table**. We then applied five decision methods to pick the best design:

- **Naive (deterministic)** — just pick the cheapest design for the average scenario. Baseline / benchmark.
- **Expected-value (stochastic)** — pick the design that minimises the probability-weighted average cost. Better than naive but still gambles on low-probability peaks.
- **Minimax-regret** — pick the design whose worst regret (how much worse than optimal could we look?) is smallest. Refuses to be embarrassed in any scenario.
- **Maximin** — pick the design whose worst-case outcome is best. Pure pessimist / insurance-first approach.
- **CVaR (Conditional Value at Risk)** — focuses on the average of the worst 30% of outcomes. Widely used in finance and energy for risk-averse decisions.

We then defined the **Value of Robustness (VoR)**: how much better (in cost and service) does the robust design do compared to the naive one in the worst case? Answer: the robust design saves **£87,422/yr** (61%) and lifts guaranteed service from 85% to 96% in the worst case.

5 — Every Tool Used & Why

Python 3

The programming language. Chosen because it is free, has the best scientific libraries, and runs the full pipeline in under a minute.

scipy linprog / HiGHS LP solver

Every day of the year the chargers can be scheduled flexibly (smart charging). Finding the cheapest schedule for 24 hours is a Linear Programming (LP) problem — HiGHS solves it in milliseconds. Used to verify that our simpler greedy scheduler gives the same answer (it does).

NumPy & Pandas

Fast number crunching. 8,760 hours × 150 designs × 9 scenarios = millions of arithmetic operations. NumPy vectorises these; Pandas organises the results into tables.

Monte-Carlo simulation (500 samples)

We don't know the exact future demand, so we drew 500 random but physically realistic demand trajectories and ran the model on each one. This gives a probability distribution of outcomes — not just one number.

Bootstrap 95% confidence intervals

After the Monte-Carlo, we re-sampled the 500 results 1,000 times to calculate tight confidence bands. This tells us: 'we are 95% sure the annual cost is between £X and £Y'.

Sobol sensitivity analysis

A mathematical method (variance decomposition) that answers: which uncertain input (demand? solar? tariff? costs?) matters most to the final cost? Sobol indices rank every driver. Demand growth turned out to dominate.

PVGIS API

Live EU Commission solar data for Newcastle. Instead of assuming a solar factor, we downloaded the actual hourly irradiance for the site. More accurate and fully reproducible.

National Grid ESO API

Live hourly grid carbon intensity for North East England. Lets us calculate carbon savings hour by hour — a solar panel saving a kWh at noon displaces a different carbon intensity than the same kWh at midnight.

Battery degradation model (LFP chemistry)

Lithium Iron Phosphate batteries (LFP) were modelled: cycle ageing from daily discharge, calendar ageing from time, combined to estimate pack lifetime (~12 years), replacement cost, and end-of-life material recovery (~45%). LFP was chosen over NMC because it is cobalt-free and its thermal-runaway onset is ~270°C vs ~150°C — much safer for depot use.

Bass diffusion S-curve cap

Demand cannot grow exponentially forever — the model caps it with an S-curve (Bass 1969 diffusion model) so year-15 demand is physically plausible through a 15 kW grid connection.

MATLAB (charging_hub_unified.m)

A unified MATLAB script provides an alternative implementation of the same optimisation — confirming results and enabling the competition's in-person demonstration (Level 2 at Warwick, July 2026).

python-docx & reportlab

Automated document generation. The Executive Summary PDF and all figures are built by code from live results — so every number in the PDF is always correct and up to date.

PowerPoint COM automation (PowerShell)

Converts .pptx → .pdf and .docx → .pdf using the same engine as clicking 'Save as PDF' in Word/PowerPoint — pixel-perfect.

Git / GitHub

Full version control. Every change is committed with a message. The repo history proves the work was done incrementally, not dumped in one night.

6 — File-by-File Guide (What Each File Does)

src/config.py

Central settings file. All prices (electricity tariff, battery cost, panel cost, discount rate) and physical constants live here. Change one number and the whole project updates.

src/demand_model.py

Reads the DfT trial data and builds hourly demand for any scenario (Low / Medium / High) and any year. Applies the S-curve growth cap.

src/pv_model.py

Reads the PVGIS solar data and scales it to the chosen panel size (kWp). Returns 8,760 hourly generation values.

src/energy_balance.py

The core simulator. Given a design (PV + battery + chargers) and a demand series, it steps through 8,760 hours: solar fills demand first, then battery, then grid (capped at 15 kW). Anything unserved is logged as lost service.

src/lp_dispatch.py

The smart-charging scheduler. Formulates and solves the LP for each 24-hour window — finds the cheapest way to charge vehicles given the ToU tariff, grid cap and battery constraints.

src/optimization.py

The search engine. Tests all 150 designs × 9 scenarios, builds the cost matrix, applies the five decision rules, finds the Pareto frontier, and re-verifies reported designs with the LP dispatcher.

src/economics.py

Calculates every cost: CAPEX (panel + battery + chargers + installation), annualised repayment, O&M, battery replacement, time-of-use grid bills, peak demand charges, export revenue, and LCOE.

src/emissions.py

Carbon accounting — both the flat-factor version and the hour-resolved version using the real National Grid ESO series. Also computes the cost-vs-carbon trade-off frontier along the PV axis.

src/battery_sustainability.py

LFP battery lifecycle analysis: cycle ageing, calendar ageing, state-of-health trajectory, second-life stationary use, end-of-life material recovery, embodied CO₂, and safety comparison with NMC.

src/uncertainty.py

Monte-Carlo engine: draws 500 correlated demand samples, runs the model on each, computes bootstrap confidence intervals, and builds the uncertainty fan chart.

src/sensitivity.py

Sobol sensitivity analysis: varies every uncertain input independently, decomposes the variance of the output, and ranks drivers.

src/visualize.py

Makes all the figures: Pareto frontier, uncertainty fan, tornado chart, Sobol indices, robustness heatmap, cost-carbon frontier, etc.

run_analysis.py

The master pipeline. Run this one file and it does everything: loads data → simulates → optimises → Monte-Carlo → sensitivity → economics → carbon → sustainability → generates all outputs → saves results.json.

scripts/build_executive_summary.py

Generates the 2-page anonymised PDF for the competition's Level-1 blind review. All numbers come from results.json — never hardcoded.

scripts/build_presentation.py

Generates the 3-minute presentation script and PowerPoint deck.

matlab/charging_hub_unified.m

All four MATLAB approaches merged into one script with a menu. Used for the in-person Warwick demonstration (Level 2, July 2026).

outputs/results.json

The single source of truth. Every number in every PDF comes from here.

REFERENCES.md

38 academic citations for every assumption — BEIS, IRENA, BloombergNEF, Harper 2019, Bass 1969, Saltelli 2010, Rockafellar & Uryasev 2000, Silvente 2018, DfT, PVGIS, etc.

7 — Key Findings (The Surprises)

Finding 1 — Smart charging beats storage.

Everyone assumed a large battery would be essential to buffer the evening peak below the 15 kW grid limit. Every single run of the optimiser chose *zero battery*. Why? Because flexible smart charging — spreading vehicle charging across the night rather than all at once — flattens the load profile below the grid cap entirely. A battery only becomes necessary when the grid connection weakens toward 10 kW (near off-grid). That boundary was then quantified.

Finding 2 — Cost and carbon are aligned, not in tension.

The cost-carbon trade-off frontier shows that going from 5 kWp to 20 kWp of solar raises annual cost only marginally (£55.9k → £55.0k) while lifting carbon saving from 6% to 17%. The cost-optimal design is also near the carbon-optimal design — you don't have to choose between saving money and saving carbon.

Finding 3 — Correlated demand makes deterministic sizing dangerous.

When trips, utilisation AND growth all spike together (correlated High scenario), the deterministic (average-case) design fails by 10–14 percentage points of service. Robust methods catch this tail; expected-value methods under-weight it.

Finding 4 — The recommended robust design (numbers):

Solar panel (PV)	20 kWp — enough to cover daytime load, reasonable cost
Battery storage	0 kWh — zero! Smart charging makes it unnecessary
Charge bays	8 — enough for the fleet
Capital cost	approx £36,000
LCOE	£0.25/kWh (well below UK grid retail ~28p/kWh)
Worst-case cost saving vs naive	£87,422/yr (61%)
Worst-case service guaranteed	96% fleet charged on worst demand day
Carbon saving (hourly-resolved)	16% (4.6 tCO ₂ /yr)

8 — The Methodology Flow (Step by Step)

Step 1 — Load real data

DfT trial → monthly demand profile. PVGIS API → hourly solar. National Grid ESO → hourly grid carbon intensity.

Step 2 — Build scenarios

Scale demand into Low / Medium / High × three growth rates = 9 scenarios. Apply S-curve cap so year-15 is physically reachable.

Step 3 — Simulate 150 × 9 designs

For each design and each scenario run 8,760 hours of energy balance. Record cost, service, solar fraction, carbon saving.

Step 4 — Build the cost matrix & apply decision rules

Five methods (naive, stochastic, CVaR, minimax-regret, maximin) pick a winner. Map the Pareto frontier of cost vs robustness.

Step 5 — LP verification

Re-run the handful of reported designs under the exact LP-optimal dispatch to confirm the greedy scheduler was not biased.

Step 6 — Monte-Carlo uncertainty

Draw 500 correlated demand samples, run all, compute bootstrap 95% CIs. Plot the uncertainty fan.

Step 7 — Sobol sensitivity

Decompose variance: which uncertain input drives cost the most? Demand growth dominates; solar variability is secondary.

Step 8 — Economics & sustainability

Full CAPEX / OPEX / LCOE stack. Battery lifecycle, second-life, material recovery. Hourly carbon accounting. Cost-carbon frontier.

Step 9 — Output everything

results.json → Executive Summary PDF + Presentation PDF + 13 figures + battery_sustainability.json + lp_verification.csv.

9 — Figures Produced

01_demand_scenarios.png — Demand over the year for Low/Medium/High scenarios

02_pareto.png — Cost-vs-robustness Pareto frontier — the five decision rules

03_uncertainty_fan.png — Monte-Carlo fan — 500 demand trajectories

04_tornado.png — Tornado chart — one-at-a-time sensitivity

05_sobol.png — Sobol global sensitivity indices (first-order + total-order)

06_energy_balance.png — Hourly energy balance for the recommended design

07_carbon_intensity.png — National Grid ESO hourly carbon intensity (real data)

08_battery_degradation.png — Battery state-of-health trajectory over project life

09_lcoe_breakdown.png — Levelised cost breakdown by component

10_robustness.png — Robustness-of-robustness sweep (penalty × grid connection)

11_sensitivity_heatmap.png — Sensitivity heatmap across all uncertain parameters

12_carbon_cost.png — Cost vs carbon saving frontier along the PV axis

methodology_flow.png — End-to-end flow diagram (used in Executive Summary)

10 — Competition Context

This is the **National Competition for Sustainable e-Micromobility**, organised by the University of Warwick, IIT Kharagpur and IIT BHU Varanasi (SimLionics Pvt Ltd). **Level 1 deadline: 10 June 2026, 5 pm.** Level 2 is an in-person demonstration at Warwick on 1 July 2026.

Level 1 is judged on five equal criteria (20% each):

- Problem Understanding — do you know the real engineering problem?
- Relevance & Innovativeness — is the method novel vs standard practice?
- Approach — is the algorithm/flow sound and well-described?
- Feasibility — can this actually be built and does it make economic sense?
- Originality & Authenticity — is the work genuinely the team's own?

The project targets 19-20/20 on each of the first four criteria. The fifth (Originality) is entirely in the team's hands — the code is original, but the 2-page executive summary and the video must be written and recorded by the team in their own words.